

UNIVERSITÀ DI PISA

DIPARTIMENTO DI INFORMATICA

Corso di Laurea Triennale in Informatica

TBD

Relatori:
Patrizio Dazzi
Saul Urso
Jacopo Massa

Candidato:
Giuseppe Acocella

Anno Accademico 2026/2027

Abstract

ABSTRACT HERE.

Contents

1 Introduzione **2**

1.1 Contesto 2

1.1.1 Proliferazione LLM e Utilizzo Modelli Grandi come Servizio 2

1.1.2 Costo Inferenza su Modelli Grandi 2

1.1.3 Utilizzo SLM e Inferenza Locale 3

1.1.4 MoA come Tecnica per Recupero Accuracy 4

1.2 Problema Considerato 4

1.2.1 Assegnamento Chiamata Modello a Macchina 4

1.3 Obiettivo della Tesi 5

1.4 Struttura della Tesi 5

2 Background **6**

2.1 LLM e Inferenza Locale 6

2.1.1 Training vs Inferenza 6

2.1.2 Inferenza Locale e Inferenza come Servizio 7

2.1.3 Small Language Model (SML) - Definizione Operativa . . . 7

2.1.4 Motori di Inferenza Locale - Ollama 8

2.1.5 Scelta Modello *qwen2.5:1.5b* 8

2.2 Mixture-Of-Agents (MoA) - Architettura a Layer di Proposer e Aggregators 9

2.2.1 Composizione di Layer a Ruoli - Proposer e Aggregator . . 9

2.2.2 Intuizione alla Base della Composizione di Agenti 9

2.3 Distribuzione di Sistemi Multi-Agente 10

2.3.1 Vantaggi di un Sistema Multi-Agente Distribuito 10

2.3.2 Sfide Generali nei Sistemi Distribuiti 10

2.4 Modello a Componenti di ECLYPSE 11

2.4.1 Ray come Backend di ECLYPSE Remote 11

2.4.2 Vantaggi dell'Utilizzo di ECLYPSE per PeerMoA 12

3 Stato dell'Arte **13**
3.1 Mixture-Of-Agents (MoA) 13
3.1.1 Composizione Statica 13
3.1.2 Assenza Placement Fisico e Gestione Deploy 13
3.2 Frameworks di Orchestrazione Multi-Agente LLM 14
3.2.1 Scheduling Distribuito su Hardware Eterogeneo 15
3.3 Principi di Sistemi Distribuiti 15
3.4 ECLYPSE come Artefatto di Ricerca 16
3.5 Posizionamento della Tesi 16

4 Metodologia **18**
4.1 Architettura Generale di PeerMoA 18
4.1.1 Grafi di *Infrastructure* e *Application* 18
4.1.2 Ollama come Motore di Inferenza 19
4.1.3 Ciclo di Vita Comune di PeerMoa *Application* 19
4.2 Schema Push - LayerApplication 20
4.2.1 Topologia a Layer Statici 20
4.2.2 Servizi 21
4.2.3 Protocollo di Comunicazione tra Servizi 22
4.2.4 Strategia di Deploy 23
4.2.5 SingleAgentApplication come Caso Degenere del LayerApplication 23
4.3 Schema Pull - Pull Application 24
4.3.1 Topologia a Coda Condivisa 24
4.3.2 Servizi 25
4.3.3 Virtual Layer e Sistema a Doppio Contatore 26
4.3.4 Sfumatura Semantica tra Push e Pull 27
4.3.5 Protocollo di Comunicazione tra Servizi 27
4.3.6 Strategia di Deploy 28
4.4 Confronto Push vs Pull 29

5 Esperimenti **31**
5.1 Setup Sperimentale 31
5.1.1 Due Versioni di Applicativi Confrontate 31
5.1.2 Metriche di Valutazione 32

Chapter 1

Introduzione

1.1 Contesto

Prima di presentare l'effettivo studio e risultati degli schemi di deploy del MoA si fornisce del contesto che permetta di visualizzare l'intera immagine che lo circonda.

1.1.1 Proliferazione LLM e Utilizzo Modelli Grandi come Servizio

Negli ultimi anni gran parte dell'attenzione pubblica sull'informatica e l'intelligenza artificiale si è concentrata sull'utilizzo di un modello general purpose che fosse in grado di eseguire tanti task di categorie differenti. Questo ha portato, dal 2022 in poi, all'esplosione di modelli proprietari grandi (ad esempio Gemini, Claude, GPT) e del loro utilizzo tramite servizi hostati dai provider in questione.

Allo stesso tempo si sono sviluppati anche modelli open-weight, spesso piccoli (ad esempio Qwen, Mistral, Llama), la cui caratteristica è quella di poter essere scaricabili su macchine consumer.

1.1.2 Costo Inferenza su Modelli Grandi

L'utilizzo di grandi modelli ha portato ad ottimi risultati da un punto di vista di prestazioni ed accuracy ottenuta su benchmarks, ma allo stesso tempo sono caratterizzati da diversi svantaggi in costi:

- **Memoria**: Un modello grande, ad esempio da 70B di parametri richiede

hardware professionale specifico, non potrebbe mai girare su macchine desktop o consumer devices.

- **Calcolo**: Indicativamente il costo di calcolo di produzione di un token segue un andamento quasi proporzionale a quello dei parametri in gioco. Quindi più parametri, più memoria ma anche più potenza di calcolo necessaria per la produzione di un token.
- **Latenza**: Il time-to-first-token su modelli grandi potrebbe essere molto elevato.
- **Costo Economico**: Essendo tutto hostato dai provider che detengono i modelli ovviamente tutto viene tariffato, seguendo come unità di misura la produzione di token.
- **Costo Energetico**: Il training di modelli di grandi dimensioni comportano un consumo energetico non indifferente [7].

Questi punti motivano una ricerca di direzioni alternative a quelle di modelli grandi proprietari.

1.1.3 Utilizzo SLM e Inferenza Locale

Una possibile risposta ai problemi elencati precedentemente è l'utilizzo di Small Language Models (SML), ossia modelli nell'ordine dei 1-8 miliardi di parametri, potendoli utilizzare quindi su hardware consumer persino senza una GPU dedicata. Sono caratterizzati ovviamente da vantaggi e svantaggi:

- **Vantaggi**: Nessuna dipendenza da provider, nessuna necessità di accesso alla rete o tariffa per token, possibilità di deploy di modelli anche su edge.
- **Svantaggi**: Ovviamente un modello da 1-8 miliardi di parametri risulterà meno capace di uno da centinaia di miliardi di parametri offerto da un provider.

L'utilizzo di questi SLM si basa su specifiche tecniche abilitanti, come ad esempio la quantizzazione, ossia la precisione numerica dei pesi. Ridurre questa precisione assieme all'utilizzo di runtime leggeri come Ollama o llama.cpp permette di eseguire inferenza su devices consumer.

1.1.4 MoA come Tecnica per Recupero Accuracy

Tecniche come il MoA [8] provano quindi a mitigare il calo di prestazioni lasciando che più SLM collaborino tra loro per ottenere risultati più accurati. L'idea è quella di non riferirsi ad un singolo grande modello ma di utilizzare un orchestrazione di modelli più piccoli schierati in uno schema a ruoli. Nel caso del MoA ad esempio esiste il ruolo di *Proposer* (temperatura alta, si favorisce la diversità) ed il ruolo di *Aggregator* (temperatura bassa, sintetizza i contributi ottenuti dagli agenti precedenti).

Costo in Chiamate a Modelli Nel paper del MoA [8] non si affronta la questione del costo in chiamate al modello, infatti per raggiungere risultati in termini di accuracy simili ad un modello di frontiera vanno utilizzate in composizione decine di chiamate a modelli piccoli. Questo è un problema che non viene approcciato dal paper in questione dato che il focus sono i risultati in termini di accuracy.

1.2 Problema Considerato

Le chiamate, per essere eseguite realmente, devono girare su un hardware fisico. Nasce così la domanda di come assegnarle ad eventuali macchine disponibili.

1.2.1 Assegnamento Chiamata Modello a Macchina

Vogliamo analizzare da un punto di vista infrastrutturale l'assegnamento di una chiamata ad un modello su una specifica macchina, per poter studiare metriche come latenza E2E e throughput in base al tipo di schema scelto.

Deploy su Macchina Singola L'idea più semplice potrebbe essere quella di utilizzare una singola macchina fisica, non dovendo prendere quindi alcuna scelta sull'assegnamento modello-macchina. In questo caso throughput e latenza E2E dipenderebbero del tutto dalla macchina.

Molteplici Macchine e Modalità di Deploy Il passo successivo sarebbe quello di definire quali modelli posizionare su quali macchine, qualora si stia facendo riferimento ad un cluster di macchine. Da un punto di vista teorico possiamo immaginare due tipi di schieramento:

- **Deploy a Priori**: Si sceglie in modo predeterminato l'assegnamento macchine del cluster e lavoro da eseguire prima ancora che questo inizi. I task quindi non vengono richiesti dalla macchina ma vengono forniti ad essa in modo statico.
- **Deploy a Runtime**: L'assegnamento si determina a tempo di esecuzione del lavoro, facendo in modo che le macchine possano richiedere nuovi task quando sono in idle.

Ci aspettiamo quindi teoricamente che un deploy a priori sia caratterizzato da problemi di bottleneck, soprattutto su cluster di macchine eterogenee, dato che viene assegnata una quantità di lavoro equa a tutti i *worker*.

1.3 Obiettivo della Tesi

L'obiettivo della tesi è validare sperimentalmente un'estensione a scheduling dinamico, in stile pull con coda condivisa tra *worker*, dell'architettura Mixture-of-Agents (MoA) [8] originale, che adotta invece uno schema push a piazzamento statico. Entrambi gli schemi vengono ricostruiti come sistema a microservizi distribuibile. Questo ci lascia la possibilità di estendere l'esecuzione non solo su macchina singola come l'architettura originale ma anche su un cluster di macchine fisiche eterogenee. I due schemi quindi vengono deployati sia su macchina fisica singola che su cluster e successivamente vengono confrontati i risultati in termini di throughput e latenza E2E. L'accuracy dei due schemi viene ereditata dall'architettura originale, dato che la ricostruzione a microservizi non ne altera il comportamento. Viene quindi utilizzata come controllo di validità sperimentale ma non è parte dell'obiettivo della tesi.

1.4 Struttura della Tesi

Chapter 2

Background

2.1 LLM e Inferenza Locale

La tecnica del MoA si basa su funzionalità più a basso livello che rappresentano il core dei language model.

2.1.1 Training vs Inferenza

Sono le due fasi fondamentali per un modello:

- **Training**: Consiste nella definizione dei pesi del modello ed è molto costosa. Permette quindi la vera e propria creazione di un modello.
- **Inferenza**: Meccanismo che consiste nell'interrogazione del modello. Nello specifico consiste in una generazione autoregressiva: dato in input il prompt tokenizzato, il modello produce l'output un token alla volta, dove ciascuno dei token prodotti è condizionato da tutti quelli precedenti. Non può quindi essere generato un token $n + 1$ prima di un token n. Ogni produzione di token corrisponde quindi ad un forward pass.
  - **Risorse Consumate a Tempo di Inferenza**: Per effettuare un forward pass si eseguono diverse moltiplicazioni matriciali, quindi più parametri corrispondono a più FLOP[1] per token. Oltre al costo in termini di calcolo abbiamo anche costi in memoria proprio perché si mantiene in RAM/VRAM i pesi interessati al forward pass ed oltre a questo si tiene in memoria una struttura detta KV-cache, ossia

[1] Floating-Point Operations

una struttura dati chiave-valore che permette, in maniera efficiente, di guardare indietro a tutta la sequenza già prodotta di token.

2.1.2 Inferenza Locale e Inferenza come Servizio

Esistono quindi erogazioni diverse di modelli, questi possono girare su hardware di terzi a cui possiamo interfacciarci tramite API lasciando viaggiare le richieste in rete oppure possiamo lasciar girare i modelli sul nostro hardware. Questi tipi di erogazione hanno delle caratteristiche ben precise:

- **Latenza**: L'utilizzo di servizi remoti aggiunge un RTT[2] e un accodamento sull'infrastruttura che non possiamo in alcun modo controllare. Il locale abbatte queste problematiche ma ci limita eventualmente al throughput producibile dal nostro hardware.
- **Controllo e Dipendenza**: Il servizio impone le proprie condizioni, come costi e rate di token. Il locale lascia controllo pieno ma anche responsabilità operative.
- **Scalabilità**: Il servizio nasconde l'assegnamento di chiamata a modello a macchina fisica dietro l'API. Il locale lascia a noi la completa responsabilità di decidere come allocare le richieste sull'hardware disponibile.

2.1.3 Small Language Model (SML) - Definizione Operativa

Fino ad ora avevamo definito arbitrariamente che un SML avesse tra 1 e 8 miliardi di parametri. Questa affermazione non è in nessun modo formale ma vogliamo ancorarla come criterio operativo in questa tesi. Quindi con SML intendiamo un modello eseguibile senza un infrastruttura dedicata, ma eseguibile su macchine con GPU/CPU consumer, a differenza di modelli di frontiera che richiedono cluster di macchine specializzate.

[2]Round-Trip-Time

2.1.4 Motori di Inferenza Locale - Ollama

Un motore di inferenza locale è il software che carica i pesi di un modello già addestrato sull'hardware, in questo caso consumer, permettendo l'esecuzione dei forward pass. Il motore espone un interfaccia per interrogarlo, lasciando la possibilità di gestione di modelli tramite API. Esistono varie alternative, il sistema descritto in questa tesi si basa su *Ollama* ma esiste anche ad esempio *llama.cpp*, spesso utilizzato come backend di altri sistemi, oppure *vLLM* pensato per throughput alti. Ci sono alcune motivazioni per cui è stato scelto *Ollama*:

- Permette la gestione di modelli con un singolo comando, lasciando tutta la responsabilità ad *Ollama* e non a *PeerMoA*.
- La reale integrazione con il sistema PeerMoA avviene tramite API HTTP esposta dal modulo di *Ollama*.
- Permette l'interazione con hardware eterogeneo, che sia un backend di driver NVIDIA, ARM o AMD, senza sostanziali modifiche al sistema.
- Permette una gestione semplice del batching che effettua sulle richieste.

2.1.5 Scelta Modello *qwen2.5:1.5b*

La scelta di un modello molto piccolo come *qwen2.5:1.5b* ha due motivazioni:

- **Nodi Eterogenei nel Cluster**: Uno dei principi era quello di test di PeerMoA su un cluster eterogeneo. Questo implica che alcuni nodi del cluster potessero essere davvero poco potenti e questa caratteristica non avrebbe dovuto distorcere gli esperimenti a causa di bottleneck causati dall'utilizzo di modelli più grandi.
- **Precondizione sull'Accuracy**: Una delle precondizioni del MoA era che modelli piccoli hanno, da soli, accuracy modeste. Il MoA avrebbe quindi poco da recuperare su modelli non limitati.

2.2 Mixture-Of-Agents (MoA) - Architettura a Layer di Proposer e Aggregators

Il Mixture-Of-Agents [8] è una tecnica di raffinazione iterativa dove un prompt attraversa n layer. Ciascun layer riceve in input la query originale e l'output prodotto dal layer precedente e produce un nuovo output che alimenta il layer successivo.

2.2.1 Composizione di Layer a Ruoli - Proposer e Aggregator

Ciascun layer è un insieme di agenti, ossia istanze di LM. La configurazione è definita da una serie di layer consecutivi, in pipeline. Gli agenti possono avere uno di questi due ruoli:

- **Proposer**: Ha il compito di generare risposte candidate e varie tra loro. Questo tipo di istanza di modello è caratterizzata da una temperatura alta. Con temperatura alta si intende varianza più alta nel campionamento del token successivo. Dunque gli output saranno meno deterministici e più eterogenei.
- **Aggregator**: Ha il compito di sintetizzare le proposte del layer precedente in una singola risposta. Qui la temperatura è bassa proprio perché l'obiettivo è quello di sintesi.

È importante ricordare che un agente può ricoprire ambo i ruoli in base al punto in cui si trova nella pipeline. Un agente posizionato alla fine di un layer che non sia il primo ha la responsabilità di fornire un contributo al layer successivo che non sia solo di sintesi.

2.2.2 Intuizione alla Base della Composizione di Agenti

L'osservazione sulla base del paper originale del MoA [8] è quella che un LM[3] tende a produrre output di qualità superiore quando ha visibilità sulle proposte di altri LM. Questo accade a prescindere dalla dimensione del LM, i modelli che propongono possono essere anche singolarmente più deboli del modello che sintetizza.

[3]Language Model

2.3 Distribuzione di Sistemi Multi-Agente

Motiviamo i vantaggi di una distribuzione a microservizi di un sistema multi-agente.

2.3.1 Vantaggi di un Sistema Multi-Agente Distribuito

Esistono diverse motivazioni per cui distribuire un sistema multi-agente:

- **Scalabilità**: Un sistema multi-agente richiede molteplici istanze di modello attive contemporaneamente. Su una singola macchina si setta come tetto la disponibilità di risorse della macchina in questione. Costruendo invece un sistema a microservizi abbiamo ancora la possibilità di schierarlo su una macchina singola ma allo stesso tempo anche di aggregare più macchine e relative risorse disponibili.
- **Hardware Eterogeneo**: Un sistema a microservizi permette l'utilizzo di macchine già esistenti e diverse tra loro, senza la necessità dell'esistenza di un cluster di macchine fisiche ad hoc.
- **Edge Computing**: Permette redistribuzione di responsabilità tra eventuale macchina server centrale e macchine partecipanti al calcolo, spostandolo anche su di esse. Questo influisce anche sulla latenza di rete tra le macchine client ed un eventuale data center remoto.

Il deploy effettivo su un cluster di macchine fisiche custom non porta solo a vantaggi ma la sua ingegnerizzazione a microservizi sì. Questo perché la rappresentazione a microservizi non preclude in nessun modo il suo utilizzo su una singola macchina fisica potente.

2.3.2 Sfide Generali nei Sistemi Distribuiti

Come anticipato, il deploy su cluster non esclude svantaggi:

- **Comunicazione**: Agenti su macchine diverse devono avere la possibilità di comunicare attraverso la rete invece che in memoria locale. Questo porta alla necessità di eventuali definizioni di protocolli o utilizzo di serializzazioni non necessarie nel caso non distribuito.

- **Scheduling**: Bisogna decidere quale agente vada eseguito su quale macchina e quando. Abbiamo precedentemente citato ad esempio il deploy a priori o il deploy a runtime di agenti su macchine. Questo è un nuovo livello di difficoltà assente nel caso non distribuito.
- **Bilanciamento del Carico**: Bisogna che il carico di lavoro sia bilanciato in base alle capacità di calcolo di un nodo, dato che stiamo ipotizzando il cluster come composto da macchine eterogenee.

2.4 Modello a Componenti di ECLYPSE

Grazie al framework ECLYPSE [4] abbiamo definito quattro elementi fondamentali alla base del sistema:

1. ***Service***: L'unità atomica di microservizio di riferimento. Rappresenta un agente ed il suo comportamento è definito nel metodo *step()*.
2. ***Application***: Definisce l'interazione dei *Service* tra di loro. Una *Application* quindi è una composizione di *Services*.
3. ***Infrastructure***: Rappresentazione delle risorse fisiche (come ad esempio i nodi del cluster) disponibili su cui verrà mappato il grafo dell'*Application*.
4. ***Strategy***: Modalità di assegnamento tra *Services* e nodi dell'*Infrastructure*.

Questo modello a componenti è quindi alla base sia dello schema *PULL* che *PUSH*. La distinzione tra i due schemi è quindi realizzata in termini di implementazione di specifici *Service* e *Application*.

2.4.1 Ray come Backend di ECLYPSE Remote

Di base ECLYPSE [4] permette la simulazione di strategie di deployment di applicazioni su cluster. Questo fino a quando si effettua una normale simulazione. Esiste però anche la modalità di simulazione remota che permette l'effettiva esecuzione dell'applicazione a microservizi grazie all'utilizzo del framework Ray.

- **Ray**: Framework per calcolo distribuito in Python, pensato per semplificare l'esecuzione distribuita di programmi Python. Si basa su alcune primitive:
  - **Task**: Funzioni Python eseguine in modo asincrono da un nodo qualsiasi del cluster.

- **Actor**: Processi persistenti capaci di mantenere uno stato interno tra una chiamata e l'altra.
- **Object Store**: Memoria condivisa dai nodi del cluster che permette di scambiarsi dati in comune. Ray si occupa quindi della serializzazione di questi dati.
- **Scheduler del Cluster**: Ray gestisce un insieme di nodi e decide su quale nodo dovrà essere eseguito un determinato task.

Dunque possiamo dire che il sistema è descritto dalle primitive caratteristiche di ECLYPSE ma il motore "grezzo" di esecuzione è Ray che sotto il cofano si occupa dell'esecuzione distribuita del sistema.

2.4.2 Vantaggi dell'Utilizzo di ECLYPSE per PeerMoA

- Il framework si occupa esplicitamente di concetti come strategie di placement e sistemi a microservizi. Questo ci rende naturale lo sviluppo di applicazioni differenti da poter confrontare tra loro.
- Supporta l'esecuzione su reali macchine fisiche differenti tramite l'utilizzo di Ray.

Chapter 3

Stato dell'Arte

3.1 Mixture-Of-Agents (MoA)

La Mixture-Of-Agents [8] è definita come una pipeline di l layer, ciascuno di essi composto da n agenti LLM $A_{i,1}, \ldots, A_{i,n}$. Viene quindi generato un *Aggregate-and-Synthesize Prompt*, ossia una concatenazione testuale delle risposte e del prompt originale. Il MoA si differenzia dal Mixture-Of-Expert (MoE) [6], che opera più a basso livello su attivazioni e pesi della rete: Il MoA opera invece esclusivamente a livello di prompt e testo generato, quindi la composizione di agenti avviene solo tramite testo concatenato. Come già presentato nel capitolo precedente, anche qui, il MoA [8] si avvale di due ruoli funzionali, ossia *Proposer* ed *Aggregator*.

3.1.1 Composizione Statica

Nel setup del paper originale del MoA [8] si usano 3 layer fissi, lo stesso insieme di modelli open-weight usati come proposer in ogni layer e con *Qwen1.5-110B-Chat* in chiusura come *Aggregator*. Questo setup di modelli è deciso a tempo di configurazione pre-run e non modificato successivamente. Non esiste dunque alcun meccanismo che permette di modificare il setup in termini di layer o agenti in base al carico di lavoro o risultati intermedi.

3.1.2 Assenza Placement Fisico e Gestione Deploy

Il paper del MoA [8] non parla mai dell'infrastruttura fisica su cui si appoggia o del setup lato deployment: Viene solo detto che l'inferenza viene eseguita tramite l'utilizzo di un endpoint. Il paper non specifica se e come i componenti siano

distribuiti, perché il deployment fisico non è affatto modellato come variabile. È proprio in questo spazio che si inserisce in contributo di questa tesi: Introdurre la nozione di placement in un'architettura che in origine non aveva alcun concetto di deployment fisico.

Distributed MoA Un lavoro affine è quello di *Distributed Mixture-of-Agents for Edge Inference with Large Language Models* [5] che estende MoA ad una rete di n utenti edge, ciascuno con il proprio LLM locale. La comunicazione tra gli utenti avviene tramite un algoritmo detto *gossip*: Ogni utente inoltra il prompt a k vicini casuali ed aggrega le risposte ad ogni M round. Si pone l'obiettivo di trovare un buon trade-off tra accuracy e buoni risultati in latenza. Si differenzia da questo lavoro per i seguenti motivi:

- Definisce un grafo di utenti, non una ricostruzione a microservizi dell'applicativo MoA originale.
- Utilizza il *gossip* come meccanismo di comunicazione tra gli agenti.
- Non si occupa di un confronto tra applicativi *Push* e *Pull* su cluster fisicamente eterogeneo.

Time-To-First-Token Il paper, nella sezione *Limitations*, indica come l'aggregazione iterativa causi un *Time-To-First Token* alto, dato che per emettere il primo token il prompt deve prima attraversare tutta la pipeline.

Dimensione Modelli Notiamo che nel paper originale non si utilizzano SLM come da definizione operativa di questa tesi ma lo stesso tipo di modelli open-weight di dimensioni molto più grandi, come ad esempio *Qwen1.5-110B-Chat*.

3.2 Frameworks di Orchestrazione Multi-Agente LLM

Esistono frameworks che si occupano interamente dell'orchestrazione di agenti LLM. Queste permettono coordinamento da un punto di vista applicativo, come ordine di comunicazione o stato condiviso. Ne citiamo degli esempi:

- **AutoGen [9]**: Tool sviluppato da Microsoft che permette la modellazione di un flusso di agenti come entità che comunicano tra loro tramite lo scambio

di messaggi. L'agente è un oggetto Python che contiene una chiamata a modello via API.

- **CrewAI** [**2**]: Orchestrazione di agenti rule based. Gruppi di agenti sono detti *crew* ed esiste un processo di assegnamento di task sugli agenti della *crew*.
- **LangGraph** [**3**]: Rappresentazione di gruppi di agenti come grafo diretto.

Tutti e tre i framework elencati sopra operano a livello di logica applicativa delegando l'esecuzione ad un API remota. Nessuno di questi tratta il deploy dell'agente su macchina fisica come primo interesse.

3.2.1 Scheduling Distribuito su Hardware Eterogeneo

L'argomento d'interesse di questi framework non è di natura fisica o di schieramento. Quello che fanno è un esecuzione asincrona delle chiamate API. Quindi concetti di distribuzione esistono ma in termini di utilizzo di specifici endpoint. Non viene dichiarata alcuna topologia fisica, è un gap concettuale voluto proprio perché l'interesse dei framework elencati è differente.

3.3 Principi di Sistemi Distribuiti

A sostegno della policy in Pull, o deploy a Runtime, presentata in questa tesi citiamo della letteratura e principi fondamentali dei sistemi distribuiti in relazione all'attuale contesto del PeerMoA.

- **Work Stealing** [**1**]: Un classico pattern del calcolo distribuito: Invece di lasciare la responsabilità di dispatching ad un sistema centralizzato si distribuisce questa su di essi. Ogni worker mantiene una coda locale e quando è in *idle* prende task dalle code altrui o da una coda condivisa.
- **Bottleneck e Assegnamento Statico**: In assegnamento statico un worker più lento ottiene la stessa quantità di lavoro di un worker performante, lasciando gli altri in attesa di quello lento. Con un meccanismo di pull ogni worker richiede altro lavoro solo quando è libero e in questo modo il sistema si autobilancia naturalmente in base alla velocità di ciascun nodo. In questo modo non abbiamo la necessità di conoscere a priori le capacità di ciascuna macchina. Questo giustifica perché su un cluster eterogeneo ha senso testare una configurazione in PULL.

- **Pattern Pull e MoA**: Abbiamo motivato perché in un cluster eterogeneo sia adeguato l'utilizzo di uno schema PULL. Ora però è necessario contestualizzarlo nello specifico ai layer ed agenti del MoA. L'applicazione della policy in PULL ed il suo confronto con l'originale PUSH è l'ipotesi d'interesse di questo lavoro.

3.4 ECLYPSE come Artefatto di Ricerca

Il framework ECLYPSE [4] su cui abbiamo costruito PeerMoA si basa su specifici criteri:

- **Cloud-Edge Continuum**: Avvicinare il calcolo ai client per migliorarne le prestazioni, gestendo risorse e placement dei servizi in condizioni di rete e di ambienti che cambiano nel tempo.
- **Rapid Prototyping**: ECLYPSE [4] è uno strumento di ricerca per la sperimentazione rapida di sistemi a microservizi. Non si pone quindi come uno standard da usare in produzione ma come framework di prototyping di applicazioni distribuite.
- **Setup Simulazione/Emulazione**: ECLYPSE [4] lascia la possibilità di poter passare da modalità simulate a realmente eseguite (o ibride) di applicazioni a microservizi. Questo nel nostro caso ci ha permesso all'inizio dello sviluppo di prototipare l'applicazione prima ancora di lasciarla realmente girare su un cluster di macchine.

Quindi l'utilizzo di ECLYPSE [4] ci permette di ottenere allo stesso tempo la possibilità di eseguire il tool su un cluster distribuito e la possibilità iniziale di poter prototipare simulandone l'esecuzione.

3.5 Posizionamento della Tesi

Dato tutto il contesto attuale definiamo la posizione effettiva della tesi sulla base delle informazioni citate fino ad adesso:

- **Gap**: Fino ad adesso abbiamo discusso strumenti e paper che orbitavano attorno al topic di PeerMoA, ma:

  - Il MoA originale non discute di deploy fisico, rimanendo silente da un punto di vista dell'infrastruttura proprio perché non è topic d'interesse di quel paper.
  - I framework di orchestrazione multi-agente attuali (AutoGen, CrewAI, LangGraph) permettono la definizione di una coordinazione logica tra agenti ma anch'essi delegano l'esecuzione fisica ad endpoint remoti. Nessuno di questi permette di modellare un'infrastruttura fisica su cui distribuire del carico di lavoro.
  - Lo scheduling pull-based, pattern ben consolidato nella letteratura del calcolo distribuito in generale, motiva l'interesse della nostra tesi a volerlo applicare ai layer della pipeline del MoA.

- **ECLYPSE**: Il framework che ha reso possibile l'implementazione di PeerMoA che colma i tre gap citati sopra, lasciando la possibilità di ricostruire i layer del MoA cone sistema a microservizi realmente eseguibile su un cluster fisico.

- **Intersezione tra Gap**: La tesi si pone proprio tra questi tre punti:
  - Il MoA originale non ha alcuna nozione di placement, PeerMoA si occupa anche dello schieramento su macchina fisica.
  - PeerMoA permette lo schieramento distribuito su hardware fisico eterogeneo che i framework di orchestrazione moderni non gestiscono.
  - PeerMoA fornisce uno scheduling in pull che esiste esiste in letteratura ma non è mai scesa nel dominio del MoA.

- **Distribuzione MoA**: Questa tesi ricostruisce MoA come sistema a microservizi distribuibile con due schemi di deploy: push, statico e fedele alla semantica originale, e pull, dinamico, fornendone la prima validazione sperimentale in termini di throughput e latenza E2E, su un cluster fisico eterogeneo.

Da adesso in poi descriviamo quindi, in termini di implementazione, come questa ricostruzione del MoA sia stata realizzata.

Chapter 4

Metodologia

Descriviamo in dettaglio l'architettura e tutte le configurazioni di PeerMoA.

4.1 Architettura Generale di PeerMoA

Avviare un run di PeerMoA consiste nell'istanziare quattro oggetti delle seguenti classi: *Infrastructure* che rappresenta l'infrastruttura di nodi fisici, *Application* ossia il grafo composto dai microservizi, *DeployStrategy* che mappa il grafo applicazione al grafo infrastruttura e *SimulationConfig* che permette la definizione dei parametri del run. Successivamente vengono passati questi quattro oggetti ad un *Simulation* che gestisce il ciclo di vita della simulazione stessa.

Diverse Applicazioni Scaffolding Condiviso Presenteremo tre tipi di Application, ossia SingleAgentApplication, LayerApplication e pullApplication. Da un punto di vista applicativo risultano essere completamente diverse ma la loro gestione si basa in modo identico sulle istanze di oggetti appena menzionate.

4.1.1 Grafi di *Infrastructure* e *Application*

Abbiamo scelto di definire Infrastructure e Application come grafi paralleli e isomorfi, quindi per ogni servizio *XNodeService* esiste un nodo fisico *XNode*. Questo perchè ECLYPSE [4] lascia la possibilità di differenziare tra grafo applicativo composto da microservizi e grafo che deve rappresentare le risorse hardware disponibili.

4.1.2 Ollama come Motore di Inferenza

Tutti i servizi che i occupano di effettuare interrogazioni a modello lo fanno interfacciandosi ad ollama tramite il suo modulo python ed il metodo `ollama.AsyncClient().chat()`. Quindi in tutti gli schemi il layer di inferenza è lo stesso.

PeerMoA e Configurazione da CLI Una delle caratteristiche di PeerMoA è quella di lasciare la configurazione a tempo di lancio del run. Ci permette di selezionare specifiche caratteristiche come:

- Modello utilizzato, acquisito dai modelli pullati tramite Ollama.
- Topic MMLU del dataset su cui effettuiamo il benchmark.
- Tipo di *Application* che vogliamo runnare.
- Eventuale numero di worker in caso di *Pull Application*.
- Policy utilizzata sulla coda in caso di *Pull Application*.

4.1.3 Ciclo di Vita Comune di PeerMoa *Application*

In tutti gli schemi di *Application* il ciclo di vita del run di PeerMoA segue specifiche fasi comuni:

1. **Bootstrap**: Si apre il run e si carica il dataset MMLU del topic selezionato, trasformandolo in prompt ed immettendolo con qualche sistema di backpressure[1].
2. **Query a Modello**: Si effettua una chiamata a modello.
3. **Scoring e Terminazione**: Si accumulano le risposte, si calcola lo score e le metriche temporanee ed eventualmente si calcolano le metriche finali lanciando *SIGINT* in caso di ultimo prompt.

Queste fasi sono distribuite su microservizi differenti che hanno ciascuno di essi la responsabilità sulle operazioni in merito.

[1]Entrambi i sistemi limitano volontariamente la quantità di entry sospese nel sistema per evitare preventivamente utilizzo eccessivo di memoria.

4.2 Schema Push - LayerApplication

Descriviamo la prima delle due *Application*, ossia la nostra versione a microservizi della Mixture-Of-Agent [8] originale.

4.2.1 Topologia a Layer Statici

Il *LayerApplication* viene generato sulla base di due parametri, ossia `num_layers` chiesto da CLI e `num_agents_per_layer` di default 3 e non chiesto da CLI. Per ogni layer `i` vengono creati i seguenti servizi:

- `InputNodeService-L{i}` in ingresso al layer.
- `MidNodeService-L{i}` in quantità `num_agents_per_layer`.
- `OutputNodeService-L{i}` in uscita al layer.

All'esterno dei layer, quindi in cima ed in fondo all'intera pipeline vengono posizionati invece

- `HeadBenchmarkService` in ingresso alla pipeline.
- `TailBenchmarkService` in uscita alla pipeline.

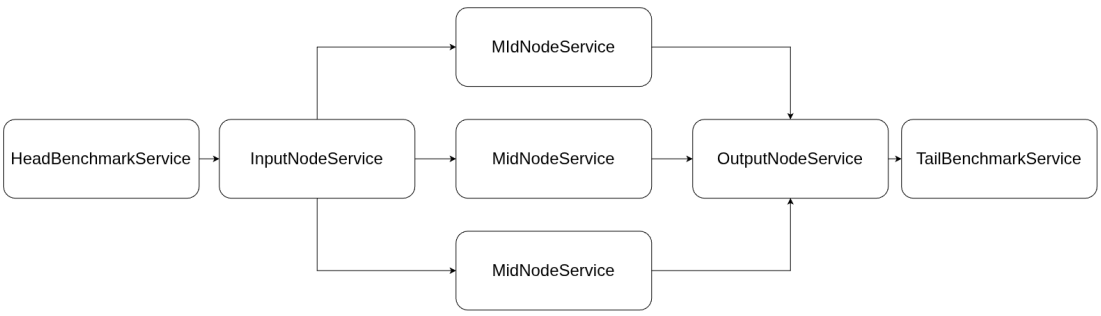


Figure 4.1: Rappresentazione di una pipeline composta da un singolo layer.

In questo schema quindi ciascun servizio è fissato per ogni layer ed esisterà come istanza di oggetto Python per l'intera durata del run. Non esiste nessun tipo di concetto di pool di worker intercambiabile, ciascun servizio ha dipendenze sui servizi precedenti.

4.2.2 Servizi

Descriviamo dettagliatamente tutte le nostre sottoclassi della classe base Service offerta da ECLYPSE [4].

`HeadBenchmarkService` Si occupa della gestione del dataset, nello specifico del `dataset_test` da dove acquisiamo i prompt MMLU[2] da fornire al modello. Successivamente gestisce i contatori `max_in_flight` e `current_in_flight` rispettivamente dimensione massima della finestra di prompt in circolazione e quantità corrente. Questo sistema di backpressure ed ACK è stato implementato per controllare eventuali problemi di OOM[3]. La dimensione della finestra è arbitrariamente fissata a 10 messaggi. Il microservizio popola quindi il `messageDict` con l'inizializzazione dei suoi campi e lo invia al successivo `InputNodeService`.

`InputNodeService` Servizio molto semplice che si occupa della ritrasmissione in broadcast del `messageDict` ai successivi `MidNodeService`. Definito più per poter ben identificare un'entità di layer.

`MidNodeService` Svolge il ruolo di *Proposer*, essendo caratterizzato da una temperatura alta (0.7) a tempo di chiamata di modello. Assume due tipi di comportamento differenti, infatti:

- Se è il primo layer della pipeline genera una risposta testuale senza nessun contributo precedente.
- Se non è il primo layer allora genera testo sulla base del contributo testuale del `OutputNodeService` precedente.

Va quindi puntualizzata l'idea di "visibilità sulle proposte altrui", menzionata nel capitolo del background. La visibilità è tra layer, non tra agenti del layer stesso. Nello specifico il contributo testuale precedente è definito nel campo `generatedText` del `messageDict`. Infine questo servizio invia il `messageDict` al `OutputNodeService` del suo layer.

`OutputNodeService` Svolge il ruolo di *Aggregator* attendendo un numero di messaggi pari alla quantità di `MidNodeService` per layer. Bufferizza i messaggi in un dizionario `message_map` indicizzato sull'id che permette la correlazione di risposte

[2]Massive Multitask Language Understanding
[3]Out-Of-Memory

di `MidNodeService` alla stessa entry del dataset. Anche questo servizio si comporta in due modi differenti in base al suo posizionamento: Produce sempre un `char-synth` con temperatura bassa (0.1) a tempo di chiamata di modello. Se non è nell'ultimo layer produce anche testo a temperatura alta (0.7) per popolare il campo `generatedText` del `messageDict` come contributo al layer successivo.

`TailBenchmarkService` Si occupa della gestione delle metriche[4] e dei contatori per l'invio dell'ACK al `HeadBenchmarkService` per la gestione della finestra dei messaggi in volo. Oltre a questo calcola anche lo score controllando il `char-synth` prodotto dal `OutputNodeService` con l'originale risposta attesa contenuta nel campo `expectedAnswer` del `messageDict`. A tempo di ricezione messaggio relativo all'ultima entry del dataset si occupa dell'invio di *SIGINT* per la terminazione del run.

4.2.3 Protocollo di Comunicazione tra Servizi

In questo schema i servizi comunicano tra loro in MPI[5], ossia utilizzando primitive di `mpi.send()`, `mpi.recv()` e `mpi.bcast()`. È importante denotare che il metodo `mpi.recv()` è bloccante, dunque ferma l'esecuzione del metodo `step()` del microservizio, in attesa della ricezione di un messaggio.

`MessageDictTemplate` Definisce il formato del messaggio che circola per ciascuna delle entry del dataset.

```
def get_message_dict_template():
    messageDict = {
        "id": 0,
        "originalPrompt": "",
        "expectedAnswer": "",
        "generatedText": "",
        "predictedAnswer": "",
        "layerPredictions": [],
        "count": 0,
        "start_time": 0
    }
    return messageDict
```

Il messaggio contiene informazioni riguardanti la entry del dataset come ad esempio: l'id, il quesito-prompt del dataset MMLU, la risposta corretta, il testo generato come contributo per i layer successivi, la risposta generata dall'*Aggregator* e la cronologia delle risposte date per il calcolo dello score.

[4]Throughput e Latenza E2E
[5]Message Passing Interface

ACKMessage Esiste anche un altro messaggio minimale, definito a runtime, che permette la gestione della backpropagation tra `HeadBenchmarkService` e `TailBenchmarkService`. Questo segue comunque uno stile di comunicazione MPI ma viaggia su uno specifico arco presente tra il servizio in cima ed il servizio in coda alla pipeline.

4.2.4 Strategia di Deploy

Si definisce in modo fisso, a priori, la mappatura tra *Service* e *Node* tramite un dizionario

```
def get_static_layer_deploy_strategy(num_layers=1, num_agents_per_layer=3):
    mappingServiceToNode = {
        "HeadBenchmarkService": "HeadBenchmarkNode",
        "TailBenchmarkService": "TailBenchmarkNode"
    }

    for i in range(1, num_layers + 1):
        mappingServiceToNode[f"InputNodeService-L{i}"] = f"InputNode-L{i}"
        mappingServiceToNode[f"OutputNodeService-L{i}"] = f"OutputNode-L{i}"
        for j in range(1, num_agents_per_layer + 1):
            mappingServiceToNode[f"MidNodeService-L{i}-({j})"] = f"MidNode-L{i}-({j})"

    return StaticStrategy(mapping=mappingServiceToNode)
```

Si itera quindi sul valore di `num_layers` e `num_agents_per_layer` e si assegna a priori il *Service* ad un *Node* dell'*Infrastructure*. Si sta quindi mettendo in pratica l'isormorfismo tra grafi *Application*/*Infrastructure* citato precedentemente, dichiarato così a tempo di configurazione.

4.2.5 SingleAgentApplication come Caso Degenere del LayerApplication

Per dare la possibilità di poter definire un run in schema push senza tutta la pipeline si definisce la *SingleAgentApplication* che schiera in una catena lineare a 4 nodi alcuni servizi prima menzionati.

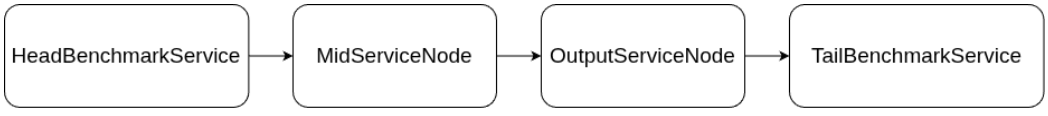


Figure 4.2: Rappresentazione di SingleAgentApplication.

I punti da analizzare di questa architettura sono due:

- La denominazione "*SingleAgentApplication*" potrebbe risultare fuorviante perchè di chiamate a modello se ne effettuano in realtà due: *MidNodeService* chiama Ollama a temperatura alta da *Proposer* e *OutputNodeService* lo fa a temperatura bassa da *Aggregator*.

- L'esistenza di questa *Application* dimostra che la reingegnerizzazione a microservizi permette lo schieramento e la definizione arbitraria di nuove applicazioni sulla base di servizi con compiti ben distinti. Definiamo questo caso infatti come degenere perchè non viene effettuata alcuna mixture. Essa non introduce nuovo codice, è solo la definizione di una nuova *Application* come composizione di *Service* già esistenti.

Questo tipo di setup non è volontariamente riportato nei risultati dei benchmark effettuati successivamente, è solo una baseline di riferimento per poter effettuare un task senza mixture.

4.3 Schema Pull - Pull Application

Descriviamo la seconda Application, che propone uno schema differente rispetto a quello originale della Mixture-Of-Agent [8].

4.3.1 Topologia a Coda Condivisa

La topologia di questo schema si basa sull'esistenza di due servizi `QueueHeadService` e `SynthOutService` rispettivamente in cima ed in fondo al grafo. Al centro del grafo è presente un pool di una quantità non predefinita di worker, rappresentati da istanze della classe `ElabMidService-{i}`.

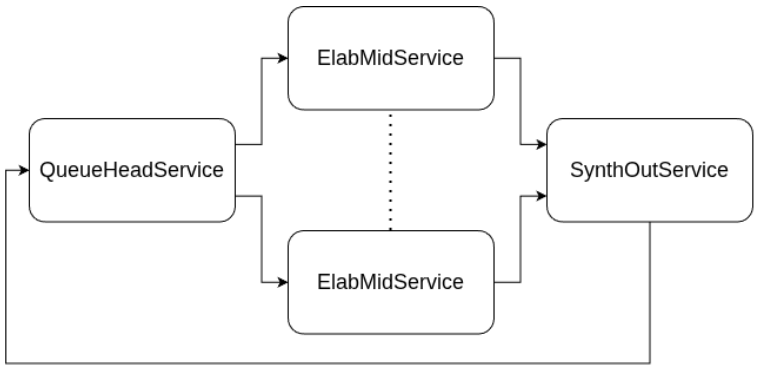


Figure 4.3: Rappresentazione di una Pull Application.

Ciascuno dei worker è collegato in modo bidirezionale a `QueueHeadService` e `SynthOutService` per poter effettuare richieste REST ad entrambi i servizi. I due *Service* alle estremità del grafo sono a loro volta collegati da un arco per poter permettere l'implementazione dei virtual layer.

Virtual Layer Prima ancora di scendere nei dettagli implementativi di ciascun servizio possiamo immaginare che un singolo attraversamento di questo grafo `QueueHeadService` → `ElabMidService-{i}` → `SynthOutService` corrisponde ad un path in un layer fisico dello schema *Push*. Dunque un messaggio proposto da un `ElabMidService` ed aggregato dal `SynthOutService` per meno di tre volte deve essere rispedito nella coda del `QueueHeadService`. Definiamo quindi un concetto di virtual layer, gestito all'interno del messaggio che circola. Nello specifico viene mantenuto un contatore che viene incrementato e controllato ad ogni giro dal `SynthOutService` per valutare se rispedire indietro la entry o se lasciarla uscire.

Worker Proposer In questo schema una entry viene acquisita ed elaborata da un worker a prescindere da chi se ne sia occupato nel virtual layer precedente. Questo si basa su una richiesta *GET* da parte del `ElabMidService` worker all'endpoint esposto dal `QueueHeadService`.

4.3.2 Servizi

Descriviamo anche in questo caso tutte le nostre sottoclassi della classe base *Service* e *RestService* offerta da ECLYPSE [4].

`QueueHeadService` Servizio di tipo REST che si occupa di mantenere una coda di dimensione prefissata a 25. È l'equivalente dei messaggi `max_in_flight` nello schema *Push*. A differenza dei normali *Service* questo è sottoclasse di RESTService, dunque il suo comportamento è definito dai seguenti metodi:

- `on_deploy` carica lo split del benchmark MMLU e pre-riempie la coda fino alla dimensione `target_queue_size`.
- Endpoint *GET* `/pull` permette l'estrazione dalla testa della coda di una entry e avvia lo `start_time` del messaggio se è la sua prima estrazione. Restituisce un messaggio speciale *EMPTY* in caso di coda vuota.
- Endpoint *POST* `/requeue` definisce il core della politica di reinserimento messaggi in coda (*TAIL* o *HEAD*). Un messaggio prima di essere consegnato al `SynthOutService` deve essere popolato di 3 elaborazioni di *Proposers*,

dunque deve essere reinserito in coda se questa condizione non è rispettata. A tempo di avvio di run in *Pull* viene lasciata la possibilità di scelta tra due politiche:

  - *HEAD* dà priorità ai task già in lavorazione, facendoli terminare prima. Questo può però lasciare la possibilità ai worker di ripescare immediatamente ciò che hanno appena rimesso in coda creando potenziali starvation su altri worker.
  - *TAIL* rompe questo schema non dando però priorità ai task già in lavorazione. Questo potrebbe quindi influire negativamente sulla latenza E2E.

`ElabMidService` Servizio worker con stile di comunicazione REST. A differenza delle altre due classi di *Service* non sono server REST che espongono endpoint ma sono client che li invocano. Nello specifico il loro metodo `step()` si basa su queste azioni:

- Si effettua il *PULL* del messaggio invocando l'endpoint `/pull`.
  - Se la coda è vuota si attende.
  - Se si ottiene un task si invoca la chiamata a modello, accodando il risultato nel campo `elaborations` del messaggio.
- Si verifica, in base al campo `count` del messaggio, se instradare il messaggio al `SynthOutService` o se restituirlo alla coda del `QueueHeadService`.

`SynthOutService` Servizio REST che espone un singolo endpoint `/synthesize`. Questo gestisce un meccanismo interno e asincrono che disaccoppia la ricezione dal lavoro di inferenza con Ollama.

4.3.3 Virtual Layer e Sistema a Doppio Contatore

Il virtual layer è gestito da due contatori mantenuti all'interno del messaggio:

- `count` tiene conto del progresso all'interno del virtual layer corrente. Va quindi da 0 a `max_elaborations` e viene azzerato ad ogni transizione di layer.
- `layer_count` che rappresenta l'indice del virtual layer, viene incrementato e controllato solo da `SynthOutService`. In base al valore di questo contatore si possono eseguire due operazioni diverse:

– Se `layer_count < max_layers - 1` allora non è l'ultimo layer. Va generata una stringa da *Proposer* che viene posizionata nel campo `previous_layer_synth`, vengono azzerati i contatori `elaborations` e `count` e infine si incrementa il contatore `layer_count` riposizionando il messaggio in coda.

– Se `layer_count == max_layers - 1` allora è l'ultimo layer. Si può quindi calcolare il `char-synth` e calcolare le metriche.

4.3.4 Sfumatura Semantica tra Push e Pull

Va menzionata una differenza all'interno dei layer tra i due schemi a livello semantico. Nello schema *Push* la generazione di un *Proposer* di un layer è indipendente dagli altri suoi pari. Ogni contributo infatti viene aggregato solo al raggiungimento del `OutputNodeService`. Nello schema *Pull* invece un virtual layer è realizzato come tre passaggi sequenziali sullo stesso task. Di conseguenza ogni worker vede i contributi precedenti appertenenti a *Proposer* interni al virtual layer. È un'approssimazione da un punto di vista semantico prodotto dall'utilizzo della coda condivisa. Anche per questo teniamo conto in ogni caso anche dell'accuracy come baseline di confronto per non compromettere l'equivalenza comportamentale dei due schemi.

4.3.5 Protocollo di Comunicazione tra Servizi

A differenza dello schema *Push* lo schema *Pull* non si basa sul MPI ma sulle interfacce REST. Ogni comunicazione è definita come chiamata ad un endpoint REST esposto dai servizi `QueueHeadService` e `SynthOutService`. Dunque la topologia infrastrutturale rappresenta una possibilità di comunicazione mentre l'instradamento effettivo è deciso interamente a livello applicativo.

Il formato del `messageDictTemplate` varia leggermente dovendo mantenere lo stato interno e l'indicizzazione dei virtual layer.

Si notano quindi i campi per mantenimento stato e indicizzazione layer e l'array per le elaborazioni prodotte internamente ai virtual layer.

```
    def get_message_dict_template_pull():
        messageDict = {
            "id": 0,
            "originalPrompt": "",
            "expectedAnswer": "",
            "elaborations": [],
            "predictedAnswer": "",
            "count": 0,
            "layer_count": 0,
            "previous_layer_synth": "",
            "start_time": 0.0,
            "total_expected": 0
        }
        return messageDict
```

4.3.6 Strategia di Deploy

Come per lo schema *Push* anche qui da un punto di vista di Strategy di ECLYPSE il mapping è fisso effettuato su due grafi isomorfi di *Application* e *Infrastructure*.

```
    def get_static_pull_deploy_strategy(num_workers=3):
        mappingServiceToNode = {
            "QueueHeadService": "QueueHeadNode",
            "SynthOutService": "SynthOutNode"
        }

        for i in range(1, num_workers + 1):
            mappingServiceToNode[f"ElabMidService-{i}"] = f"ElabMidNode-{i}"

        return StaticStrategy(mapping=mappingServiceToNode)
```

Va ben chiarito quindi che il piazzamento *Service* e *Node* dell'Infrastructure è statico. Ciò che risulta essere Pull o dinamico è l'assegnamento *Task* - *Service*. È dunque un assegnamento dinamico da un punto di vista applicativo tra lavoro e microservizio.

4.4 Confronto Push vs Pull

Confrontiamo le caratteristiche dei due schemi.

Dimensione	**Push**	**Pull**
Topologia	Pipeline lineare	Hub a pool di worker
Protocollo	MPI `bcast`/`recv`	REST client/server
Backpressure	ACK: `max_in_flight=10`	Coda: `target_queue_size=25`
Nozione di "layer"	Topologia (posizione fisica)	Stato messaggio (`count`/`layer_count`)
Diversità entro layer	Campionamento parallelo indipendente	Affinamento sequenziale incrementale
Assegnamento lavoro → worker	Fisso a priori, per tutto il run	Dinamico a runtime
Piazzamento ECLYPSE	`StaticStrategy` 1:1	Identico
Isomorfismo *Infrastructure*/*Application*	Sì	Sì
Modello/parametri LLM	qwen2.5:1.5b, $T = 0.7/0.1/0.7$	Identici
Terminazione	Tail, SIGINT via file PID	SynthOut, SIGINT via file PID
Dataset	MMLU formato a risposta ABCD	Identico
Scoring	Throughput, Latenza E2E, Accuracy	Identico

Da un punto di vista metodologico le ultime quattro righe rappesentano quello che rende legittimo il confronto sperimentale tra i due schemi presentati. Questo perchè le due implementazioni divergono solo da un punto di vista di scheduling del lavoro ma non nel compito che stanno svolgendo. Oltre a questo riusciamo anche a denotare sinteticamente la differenza tra piazzamento statico definito in `StaticStrategy` e assegnamento dinamico del lavoro ai worker. L'idea quindi è quella che le due *Application* implementano la stessa semantica MoA con due

meccanismi di scheduling differenti:

- *Push* lega staticamente il ruolo logico ad un unità di esecuzione fisica per l'intera durata del run.
- *Pull* disaccoppia il task dal suo esecutore e lo scheduling viene effettuato durante il run.

Questa differenza su un cluster omogeneo di macchine potrebbe non portare a grandi differenze. Su cluster eterogeneo idealmente dovrebbe permettere un equilibrio nativo nel carico di lavoro sulle macchine.

Chapter 5

Esperimenti

Dopo la descrizione dell'implementazione degli schemi *Push* e *Pull* si passa alla loro validazione sperimentale. Definiremo prima le condizioni in cui avviene il confronto strutturale tra le due applicazioni. Successivamente si presenteranno i risultati ottenuti dalle 6 configurazioni definite e infine si discuteranno i risultati ottenuti.

5.1 Setup Sperimentale

Fissiamo tutte le variabili sperimentali necessarie a rendere legittimo il confronto tra i due applicativi *Push* e *Pull*.

5.1.1 Due Versioni di Applicativi Confrontate

Le due versioni confrontate in questo capitolo sono:

- `layerApplication`: Ricostruzione a microservizi dell'applicativo MoA originale.
- `pull_layerApplication`: Estensione basata su scheduling dinamico proposta da questa tesi.

L'obiettivo è dunque quello di validare sperimentalmente se l'estensione *Pull* migliori throughput e latenza E2E rispetto allo scheduling statico proposto in origine, testando gli schemi sia su macchina singola che su cluster eterogeneo. Il confronto risulta legittimo perchè le due *Application* divergono solo da un punto di vista di scheduling del lavoro ma non nel task complessivo che svolgono.

5.1.2 Metriche di Valutazione

Le metriche vengono definite allo stesso modo tra i due schemi *Push* e *Pull*.

Latenza End-To-End (E2E) Per ogni messaggio la latenza E2E è definita come tempo trascorso tra ingresso ed uscita dal sistema.

$$e2e_latency = current_time - start_time$$

Oltre al meccanismo generale di calcolo di latenza media, comune in entrambi gli schemi, va menzionata un'asimmetria metodologica dello `start_time` tra i due schemi:

- Nello schema *Push* il campo `start_time` viene scritto a tempo di iniezione del messaggio da parte del `HeadBenchmarkService`.
- Nello schema *Pull* la scrittura di `start_time` è posticipata al primo pull da parte di un worker, escludendo così il tempo di permanenza nella coda di pre-fill.

Throughput Permette il calcolo della quantità in uscita dal sistema, nello specifico ne calcoliamo due:

- **Istantaneo**: Calcolato ad ogni messaggio ricevuto come $\frac{received_count}{elapsed_time}$.
- **Finale**: Viene calcolato a fine run con $\frac{total_expected}{elapsed_time}$.

In-Flight (Little's Law) Vincoliamo concettualmente throughput e latenza tramite questa legge:

$$L = \lambda W$$

- L numero medio di elementi nel sistema.
- λ throughput.
- W latenza media.

In questo modo throughput e latenza non possono variare indipendentemente. Infatti aumentare la concorrenza ammessa potrebbe aumentare il throughput ma anche allungare la coda, influendo quindi sulla latenza. Questo concetto è strettamente correlato con i vincoli che abbiamo posto sul numero massimo di messaggi in volo nei due schemi, ossia `max_in_flight` e `target_queue_size`.

Accuracy Viene calcolata confrontando `predictedAnswer` e `elaborated_char` con `expectedAnswer` in percentuale su `total_expected`. Non rappresenta un obiettivo primario nel confronto sperimentale ma permette di effettuare un controllo di validità: Grazie a questa metrica riusciamo a stabilire se i due schemi producano risposte equivalenti in termini di qualità, facendo in modo che differenze di throughput e latenza non siano attribuibili a divergenze semantiche tra i due schemi.

Bibliography

[1] R. D. Blumofe and C. E. Leiserson. "Scheduling multithreaded computations by work stealing". In: *J. ACM* 46.5 (Sept. 1999), pp. 720–748. ISSN: 0004-5411. DOI: 10.1145/324133.324234. URL: https://doi.org/10.1145/324133.324234.

[2] crewAI Inc. *CrewAI: Framework for Orchestrating Role-Playing, Autonomous AI Agents*. https://github.com/crewAIInc/crewAI. Accessed: 2026-08-24. 2024.

[3] LangChain AI. *LangGraph*. https://github.com/langchain-ai/langgraph. Accessed: 2026-08-24. 2024.

[4] J. Massa et al. "ECLYPSE: A Python Framework for Simulation and Emulation of the Cloud-Edge Continuum". In: *Journal of Software: Evolution and Process* 38.1 (2026), e70081. DOI: 10.1002/smr.70081.

[5] P. Mitra et al. "Distributed Mixture-of-Agents for Edge Inference with Large Language Models". In: *arXiv preprint arXiv:2412.21200* (2024).

[6] N. Shazeer et al. *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer*. 2017. arXiv: 1701.06538 [cs.LG]. URL: https://arxiv.org/abs/1701.06538.

[7] E. Strubell et al. "Energy and Policy Considerations for Deep Learning in NLP". In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Ed. by A. Korhonen et al. Florence, Italy: Association for Computational Linguistics, July 2019, pp. 3645–3650. DOI: 10.18653/v1/P19-1355. URL: https://aclanthology.org/P19-1355/.

[8] J. Wang et al. "Mixture-of-Agents Enhances Large Language Model Capabilities". In: *arXiv preprint arXiv:2406.04692* (2024).

[9] Q. Wu et al. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. 2023. arXiv: 2308.08155 [cs.AI]. URL: https://arxiv.org/abs/2308.08155.